IITM PRAVARTAK
CATALYSING INNOVATION

Professional Certificate Programme in Agentic AI and Applications

Data Types, Variables, Control Flow & Functions

VIT_POC - Documents - IITM_New Updates - All Documents

Revisit the Learning So Far

Let’s take a quick look at the key ideas and concepts you have explored in your learning journey.

- How AI has evolved over time, from early rule-based systems to today’s intelligent models
- What Agentic AI is and how autonomous, goal-driven systems differ from traditional approaches
- The key limitations and advantages of current AI technologies in practical use
- Real-world examples demonstrating how Agentic AI is applied across industries
- The core components that enable Agentic AI systems to plan, reason and act independently
- How to get started with ChatGPT and Python for building simple AI-driven workflows
- The basics of writing Python scripts to automate tasks and interact with AI tools

Revisit the Learning So Far

Let’s take a quick look at the key ideas and concepts you have explored in your learning journey.

- How to prompt, debug and reflect effectively when working with ChatGPT
- The principles of prompt engineering for clearer, more accurate AI responses
- How to use AI tools to support debugging and improve problem-solving efficiency
- A group exercise designed to practise prompting, collaboration and applied AI skills
- Common pitfalls when using ChatGPT and how to avoid them
- An activity session to consolidate learning through hands-on exploration

Topics Covered

- Python Basics: Variables, Data Types & Numbers
- Python Data Structures: Building Blocks for Efficient Programming
- Lists
- Tuples
- Sets
- Dictionaries
- Python Conditional Statements

Topics Covered

- Loops in Python: Automating Repetitive Tasks
- Building a Simple Chatbot in Python
- Recap
- Python User-Defined Functions
- Example: Chatbot with Function

Python Fundamentals

Python Basics: Variables, Data Types & Numbers

Foundation for writing and understanding Python programs

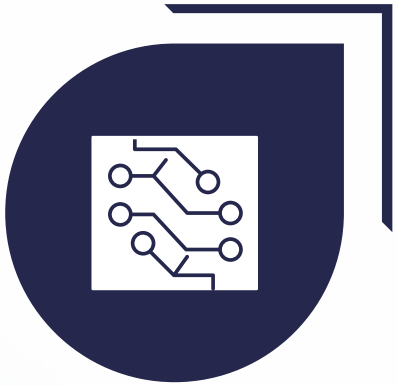
Key building blocks for data manipulation and analysis

Introduction

Emphasises simplicity and readability

Treats everything as an object

Key Concepts in Python



Variables

Data types

Numbers

Variables in Python

Definition

Store values using a name (no type declaration needed; Python is dynamically typed)

Syntax

```
x = 10

name = "Arjun"
```

Naming rules

Start variable names with a letter or underscore, not a number

Rules & Conventions for Variables

- Use letters, numbers and underscores (e.g. student_name)
- Avoid reserved keywords (e.g. for, class)
- Distinguish between cases (Age ≠ age)
- Choose descriptive names to improve readability

Built-in Data Types in Python

Numeric
Represent numbers using `int`, `float` or `complex`

Sequence
Stores ordered data using `str` `str`, `list`, `tuple`

Mapping
Link keys to values using `dict`

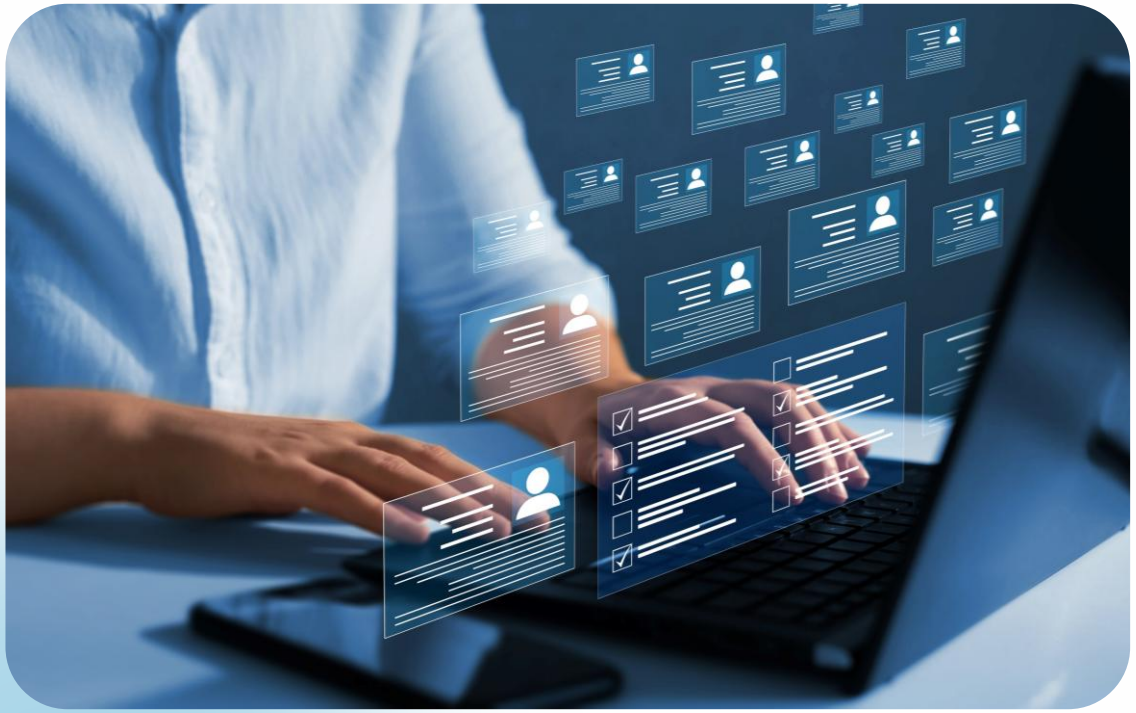
Set types
Hold unique items using set `Set` or `frozenset`

Boolean
Express truth values using `True` / `False`

None
Indicate absence of value using `None`

Checking Data Types

Use type() function:



```
x = 10

print(type(x))

y = 3.14

print(type(y))
```

Numbers in Python

Integers (int)
Whole numbers e.g., `10, -5, 200`

Floats (float)
Decimal numbers, e.g., `3.14, -2.5`

Complex numbers (complex)
$a + bj$ form, e.g., `3 + 5j`

Number Operations

Basic Arithmetic

```
a = 10
b = 3
print(a + b)
print(a - b)
print(a * b)
print(a / b)
print(a // b) # (floor division)
print(a % b) # (modulus)
print(a ** b) # (power)
```

Type Conversion

Convert Between Types Using Built-in Functions:

```
x = int(3.7) # 3

y = float(5) # 5.0

z = str(10) # "10"
```

Python Data Structures: Building Blocks for Efficient Programming

Data Structures in Python

- Organise data based on your specific needs
- Enable efficient storage, access and manipulation of data

Types of Data Structures in Python

Lists

An ordered collection where items can be added or removed

Tuples

An immutable collection, like an ID card with unchangeable personal information

Sets

A collection of unique items, like a box of toys where order does not matter

Dictionaries

A collection of key-value pairs, like a real-world dictionary, where a **word (key)** has a **meaning (value)**

Comparing Data Structures

Data structure	Characteristics
Lists	Ordered, changeable, allows duplicates
Tuples	Ordered, unchangeable, allows duplicates
Sets	Unordered, unique, no duplicates
Dictionaries	Key-value pairs, fast lookups and retrievals

Lists

Python's most versatile data structure

Introduction to Lists in Python

A Dynamic, Adjustable Collection

Ordered
Elements have a defined order, accessible by index

Mutable
Can be modified (add, remove, change items)

Versatile
Supports duplicates and mixed data types

Creating Lists in Python

Start Empty or Initialise with Items

```
# Empty listmy_list = []
# List with different data types
my_list = [1, 'apple', 3.5]
# List created using the list() constructor
numbers = list(range(1, 6)) # [1, 2, 3, 4, 5]
# Creating a list from another sequence
letters = list("Python") # ['P', 'y', 't', 'h', 'o', 'n']
```

Lists use square brackets [ ] and can contain any data type, including other lists.

Accessing List Elements

- Ordered Collections: Access items using their index (zero-based)
- Example:

```
fruits = ['apple', 'banana', 'cherry']
print(fruits[0]) # Output: apple
print(fruits[-1]) # Output: cherry
```

Modifying List Elements

Lists are mutable, allowing you to change their content without creating a new list.

```
numbers = [1, 2, 3, 4]# Change an element
numbers[1] = 9 # [1, 9, 3, 4]
# Add new item at the end
numbers.append(5) # [1, 9, 3, 4, 5]
# Insert at specific position
numbers.insert(0, 0) # [0, 1, 9, 3, 4, 5]
# Remove item with value
numbers.remove(3) # [0, 1, 9, 4, 5]
# Remove item at index
del numbers[2] # [0, 1, 4, 5]
```

Common List Operations

Built-in Methods in Python

Sorting and reversing: Use `sort()` to sort and `reverse()` to reverse the list order.

```
colors = ['red', 'blue', 'green']
colors.sort()
colors.reverse()
```

Counting and finding: Use `count()` to count occurrences and `index()` to find the index of an item.

```
numbers = [1, 2, 3, 2, 1]
count = numbers.count(2)
index = numbers.index(3)
```

Adding and extending: Use extend() to add elements from another list.

```
list1 = [1, 2, 3]
list2 = [4, 5]
list1.extend(list2)
```

Slicing Lists in Python

Create a Sublist: Use `list[start:stop:step]` to create a new list from a subset of elements.

```python
letters = ['a', 'b', 'c', 'd', 'e']

# Get elements from index 1 to 3
sliced_list = letters[1:4] # ['b', 'c', 'd']

# Get all elements from index 2 onwards
subset = letters[2:] # ['c', 'd', 'e']

# Get every second element
every_second = letters[::2] # ['a', 'c', 'e']

# Reverse a list using slicing
reversed_list = letters[::-1] # ['e', 'd', 'c', 'b', 'a']
```

Slicing creates a new list without modifying the original, preserving your data.

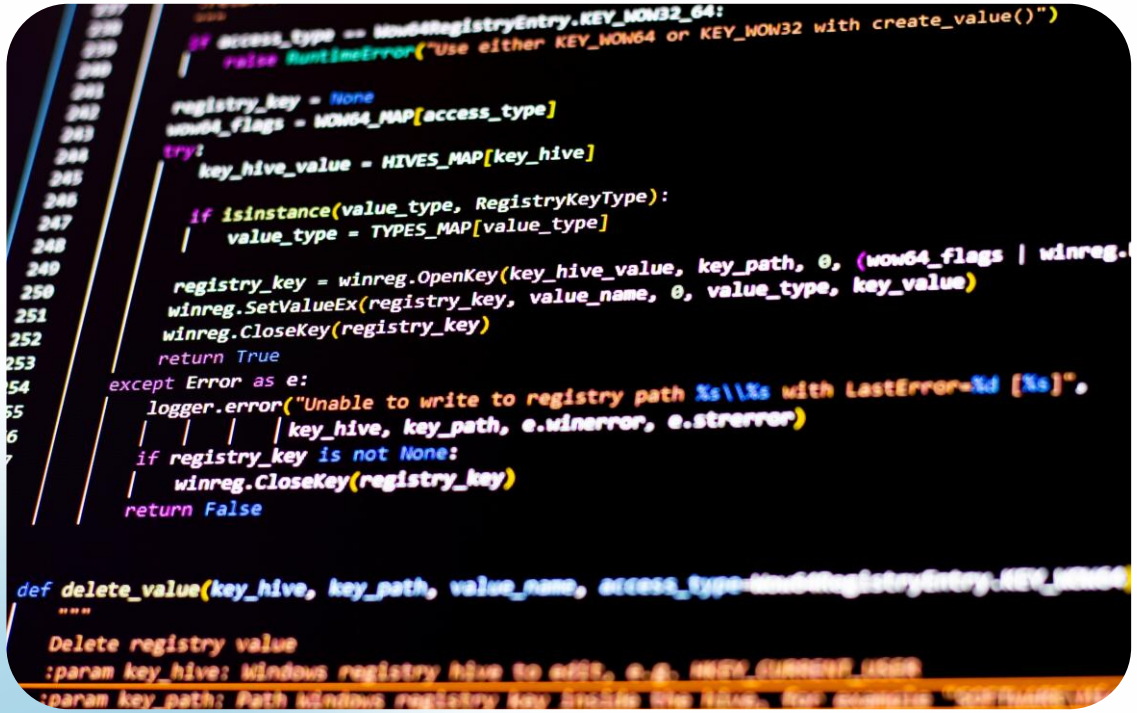
Tuples

Immutable collections for data that shouldn't change

Tuple in Python

An Ordered, Immutable Collection

Think of a tuple like an ID card. Once it's created, the information on it cannot be changed.



Ordered
Items have a specific order that does not change

Immutable
Cannot be modified after creation

Versatile
Can contain mixed data types (integers, strings, etc.)

Creating a Tuple

Creating a tuple is like creating a list, but with parentheses () instead of square brackets [].

```python
# Empty tuple
empty_tuple = ()

# Tuple with different data types
mixed_tuple = (1, 'apple', 3.5)

# Single item tuple (note the comma)
single_item_tuple = (5,) # Without comma: (5) is just an integer!

# Tuple created without parentheses (tuple packing)
coordinates = 10, 20, 30 # (10, 20, 30)

# Using the tuple() constructor
vowels = tuple('aeiou') # ('a', 'e', 'i', 'o', 'u')
```

For a single-item tuple, include a trailing comma after the value.

Key Characteristics of Tuples

Immutability

Prevents modification, addition, or removal of elements, ensuring data integrity

Dictionary keys

Can serve as dictionary keys if containing only immutable elements

Performance

Provides faster iteration and access due to fixed size in memory

Multiple return values

Allows returning multiple values from functions

Accessing Tuple Elements

Tuples are ordered like lists, allowing element access via indexing, with identical syntax.

```python
fruits = ('apple', 'banana', 'cherry')

# Access by positive index
print(fruits[0]) # Output: apple

# Access by negative index
print(fruits[-1]) # Output: cherry

# Unpacking a tuple
a, b, c = fruits
print(b) # Output: banana
```

Tuple unpacking allows assigning multiple variables simultaneously.

Tuple Operations

Tuples are immutable but support slicing, concatenation and repetition.

```python
numbers = (1, 2, 3, 4)
# Slicing
print(numbers[1:3]) # Output: (2, 3)


# Concatenation
combined = numbers + (5, 6) # (1, 2, 3, 4, 5, 6)


# Repetition
print(numbers * 2) # (1, 2, 3, 4, 1, 2, 3, 4)


# Finding min/max
print(min(numbers)) # 1
print(max(numbers)) # 4


# Count occurrences
print(numbers.count(2)) # 1
```

Tuples can't be modified directly but can be recreated from existing ones.

When to Use Tuples v. Lists

Use lists when	Use tuples when
You need to modify the collection after creation	Your data should not change (constants)
You want to add or remove items	You need dictionary keys
Your data needs to be sorted or rearranged	You want slightly better performance
You're collecting user input or growing a collection	You're returning multiple values from a function and ensuring data integrity

Sets

Set in Python

A Collection of Unique, Unordered Items

Unordered collection
Elements have no specific order and cannot be accessed by index

No duplicate elements
Duplicates are removed, and each element appears only once

Mutable
Elements can be added or removed after the set is created

Creating a Set in Python

Use curly braces { } or the `set( )` function.

```python
# Empty set
empty_set = set() # NOT {} which creates an empty dictionary


# Set with elements
fruits = {'apple', 'banana', 'cherry'}


# Set with different data types
mixed_set = {1, 'apple', 3.5}


# Creating a set from another iterable
numbers = set([1, 2, 2, 3, 4, 4]) # Duplicates removed
# Result: {1, 2, 3, 4}
```

Use set( ) to create an empty set, not { }, which creates an empty dictionary.

Key Properties of Sets

Unique Characteristics Compared to Lists and Tuples

Unordered

Elements have no specific order, making indexing or slicing impossible.

```
fruits = {'apple', 'banana',
'cherry'}
# This will raise an error:#
print(fruits[0])
```

Unique elements

Duplicates are automatically removed when creating a set.

```
numbers = set([1, 2, 2, 3, 3, 3])
print(numbers)  # {1, 2, 3}
```

Mutable with restrictions

Sets are mutable, but can only contain immutable elements.

```
# This works:
s = {1, 2, (3, 4)}
# This raises an error:#
s = {1, 2, [3, 4]}
```

Common Operations on Sets

Operations for Adding, Removing and Checking Membership

```
# Starting set
fruits = {'apple', 'banana', 'cherry'}

# Adding elements
fruits.add('orange')  # {'apple', 'banana', 'cherry', 'orange'}

# Removing elements
fruits.remove('banana')  # Error if not found

# {'apple', 'cherry', 'orange'}
fruits.discard('pear')  # No error if not found# {'apple', 'cherry',
    'orange'}

# Checking membership
print('apple' in fruits)  # True

# Length of a set
print(len(fruits))  # 3
```

Key difference: `remove()` raises a `KeyError` if the element is not found, while `discard()` does nothing.

Other Useful Methods

- `pop()` Removes and returns an arbitrary element
- `clear()` - Deletes all elements
- `copy()` - Returns a shallow copy

Mathematical Operations on Sets

Operations for Comparing and Combining Collections

Union

Combines elements from both sets:

```
set1 = {1, 2, 3}
set2 = {3, 4, 5}
print(set1 | set2)  # {1, 2, 3, 4, 5}
# OR
print(set1.union(set2))
```

Intersection

Contains elements common to both sets

```
print(set1 & set2)  # {3}
# OR
print(set1.intersection(set2))
```

Mathematical Operations on Sets

Operations for Comparing and Combining Collections

Difference

Includes elements in first set but not in second

```
print(set1 - set2)  # {1, 2}
# OR
print(set1.difference(set2))
```

Symmetric difference

Encompasses elements in either set but not both

```
print(set1 ^ set2) # {1, 2, 4, 5}
# OR
print(set1.symmetric_difference(set2))
```

Use Cases of Sets

- **Removing duplicates**

```
items = ['apple', 'banana', 'apple', 'orange']
unique_items = set(items)# Output: {'orange', 'banana', 'apple'}
```

- **Membership testing**

```
fruits = {'apple', 'banana', 'cherry'}
# Much faster than list for large collections
print('banana' in fruits)  # True
```

- **Finding unique or common elements**

```
visitors_day1 = {'Alice', 'Bob', 'Charlie'}
visitors_day2 = {'Bob', 'David', 'Eve'}
returning = visitors_day1 & visitors_day2  # {'Bob'}
all_visitors = visitors_day1 | visitors_day2
```

Sets provide O(1) time complexity for add, remove and membership tests, making them faster than lists.

Limitations of Sets

No indexing or slicing

Elements cannot be accessed by position or sliced, making sets unsuitable for ordered data.

Cannot Store Mutable Elements

Only hashable (immutable) elements can be stored, excluding lists or other sets.

Silent Operations

Methods like discard() and update()do not raise errors for missing elements, which may hide logical issues.

Dictionaries

Dictionary in Python

Key-Value Pairs for Efficient Data Storage and Retrieval

Key-value pairs

Each element consists of a key and its associated value

Unordered (Pre-Python 3.7)

Traditionally unordered, though Python 3.7+ preserves insertion order

Mutable

Can add, modify, or remove key-value pairs

Unique keys

Keys must be unique and immutable, while values can be any type

Creating a Dictionary in Python

Use Curly Braces { } or dict ( ) Function

```python
# Empty dictionary
empty_dict = {}  # OR dict()

# Dictionary with items
person = {'name': 'Alice', 'age': 25, 'city': 'New York'}

# Using dict() constructor
car = dict(make='Toyota', model='Corolla', year=2020)

# Dictionary from list of tuples
items = [('apples', 5), ('bananas', 3)]
fruit_count = dict(items)  # {'apples': 5, 'bananas': 3}

# Dictionary comprehension
squares = {x: x**2 for x in range(6)}
# {0: 0, 1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
```

Dictionary comprehension allows creating dictionaries from iterables, similar to list comprehensions.

Accessing and Modifying Dictionary Elements

- **Accessing values**

```python
person = {'name': 'Alice', 'age': 25}
print(person['name'])  # Output: Alice

# Using get() - safer if key might not exist
print(person.get('city'))  # Output: None
print(person.get('city', 'Unknown'))  # Output: Unknown
```

- **Adding & updating**

```python
# Adding a new key-value pair
person['city'] = 'New York'
# Updating an existing value
person['age'] = 26
# Update multiple values at once
person.update({'age': 27, 'job': 'Engineer'})
```

- **Removing items**

```python
# Remove a specific key-value pair
del person['city']

# Remove and return a value
job = person.pop('job') # Returns 'Engineer'

# Remove and return last inserted item
last_item = person.popitem()

# Remove all items
person.clear()
```

Built- in Dictionary Methods

```python
person = {'name': 'Alice', 'age': 25}

# Getting keys as a view object
keys = person.keys()  # dict_keys(['name', 'age'])

# Getting values as a view object
values = person.values()  # dict_values(['Alice', 25])

# Getting key-value pairs as a view object
items = person.items()  # dict_items([('name', 'Alice'), ('age', 25)]
    )

# Copy a dictionary
person_copy = person.copy()

# Create new dictionary with default values
template = dict.fromkeys(['name', 'age', 'city'], 'Unknown')
# {'name': 'Unknown', 'age': 'Unknown', 'city': 'Unknown'}
```

Built- in Dictionary Methods

View objects: `keys()`, `values()` and `items()` return dynamic views that update with dictionary changes.

```
person = {'name': 'Alice'}k = person.keys()print(k) # dict_keys
    (['name'])person['age'] = 25print(k) # dict_keys(['name', 'age']
    )# The view automatically updates!
```

Nested Dictionaries

Dictionaries can contain other dictionaries, forming a nested structure for complex data relationships.

```python
# Employee records with nested information
employees = {
'emp1': {
'name': 'Alice',
'age': 25,
'skills': ['Python', 'SQL'],
'contact': {
'email': 'alice@example.com',
'phone': '555-1234'
}

},
'emp2': {
'name': 'Bob',
'age': 30,
'skills': ['Java', 'C++'],
'contact': {
'email': 'bob@example.com',
'phone': '555-5678'

}

}

}

# Accessing nested data
print(employees['emp1']['name']) # Alice
print(employees['emp2']['contact']['email']) # bob@example.com
```

Importance of Nested Dictionaries

Represents hierarchical data

Stores complex objects

Organises related information

Works with JSON-like data structures

Importance of Nested Dictionaries

Tip

To avoid errors when accessing deeply nested dictionaries, use the get() method:

```
# Safely trying to access emp3's email
email = employees.get('emp3', {}).get('contact', {}).get('email')

print(email)   # None (since 'emp3' does not exist)
```

Limitations of Dictionaries

Key restriction: Must be unique and immutable; only immutable tuples are allowed

```
# ✅ Works: tuple as a dictionary key
d = {(1, 2): 'tuple key'}
print(d[(1, 2)])   # tuple key

# ❌ Raises TypeError: lists are mutable, hence unhashable
# d = {[1, 2]: 'list key'}

# ❌ Raises TypeError: sets are mutable, hence unhashable
# d = {{1, 2}: 'set key'}
```

Limitations of Dictionaries

Memory usage: Uses more memory than lists due to storing keys and values

Order considerations: Insertion order is preserved in Python 3.7+, but not in earlier versions

Python Conditional Statements

Conditional Statements in Python

Key Reasons

- Programmes need to make decisions
- Execute different code based on conditions
- For example:
  - If a student's score ≥ 50 → Pass
  - Else → Fail

The if Statement

```
x = 10if x > 5:
    print("x is greater than 5")
```

- The condition ($x > 5$) is evaluated
- If the condition is True, the indented code block runs
- If the condition is False, the indented code is skipped

The if-else Statement

```
x = 3
if x % 2 == 0:
print("Even number")
else:
print("Odd number")
```

- **Condition check:** Evaluate if x % 2 == 0
- **Decision point: B**ased on True/False result
- **Execution:** Run either the if block or the else block

The if-else statement gives two paths, ensuring one block always executes

The if-elif-else Statement

```
score = 85


if score >= 90:
print("Grade A")
elif score >= 75:
print("Grade B")
elif score >= 60:
print("Grade C")
else:
print("Grade D")
```

- Check conditions from top to bottom
- Execute the first block that evaluates to True
- Skip remaining checks once a match is found
- Run the else block only if all conditions are False

Nested if Statements

```
age = 20

if age >= 18:
if age < 21:
print("Young Adult")
else:
print("Adult")
else:
print("Minor")
```

- Check outer and inner conditions to build layered logic
- Risk reduced readability with deep nesting
- Prefer flattening logic using if-elif-else when possible

Conditional Expressions (Ternary Operator)

- **Standard if-else**

```
if x % 2 == 0:
result = "Even"
else:
result = "Odd"
print(result)
```

- **Ternary operator**

```
result = "Even" if x % 2 == 0 else "Odd"
```

Assign values clearly without full if-else blocks

Use the ternary operator for simple, one-line decisions

Logical Operators in Conditions

and

```
if x > 10 and x < 20:
```

Returns true only if both conditions are True

or

```
if x < 5 or x > 15:
```

Returns True if at least one condition is True

not

```
if not (x == 10):
```

Reverses the boolean value of a condition

Logical operators combine conditions for more complex decision-making.

Common Mistakes to Avoid

Using = instead of == for comparison

if `x = 5`: (assignment) vs. if `x == 5`: (comparison)

Forgetting indentation:

Python uses indentation to define code blocks, unlike languages with braces.

Overusing nested if statements:

Complex nested conditions make code harder to read and maintain.

Confusing boolean expressions

Using if `x` when you mean if `x` is not None or if `x != 0`

Loops in Python: Automating Repetitive Tasks

Purpose of Loops

The Problem: Repetition

```python
# Creating Counter- Inefficient way
print(1)
print(2)
print(3)
# ...
print(10)

# Efficient way using a loop
for i in range(1, 11):
    print(i)
```

Use loops to repeat actions efficiently without writing code multiple times.

Types of Loops in Python

for Loop

```
for item in sequence:    # code to repeat
```

- Iterates over sequences like lists, strings or ranges
- Use when the number of iterations is known

while Loop

```
while condition:    # code to repeat
```

- Repeats while a condition is True
- Use when the number of iterations is unknown

Nested Loops

```
for outer in sequence:
    for inner in another: # code to repeat
```

- Loops inside other loops
- Use when working with multi-dimensional data or complex patterns

The for Loop: Iterating Over Sequences

Iterates through a sequence, running the code block for each item

```
fruits = ["apple", "banana", "cherry"]

for fruit in fruits:
print(fruit)
```

Ideal for processing each element without using indices or counters

Using range() with for Loops

The range() Function

Generates a sequence of numbers for iteration

- range(stop) - 0 to stop-1
- range(start, stop, step) - with custom increments

Using range() with for Loops

- **Example**

```
for i in range(1, 6):        print(i)
```

This code will print numbers 1 through 5, stopping before it reaches 6

- **Output**

```
12345
```

Generates values on-demand, saving memory

The while Loop: Condition-Based Repetition

Key Functionality

```
count = 1

while count <= 5:
print(count)
count += 1
```

- Executes as long as its condition remains True.
- Requires a statement to eventually make the condition False.

Warning: Failing to update the loop condition, like omitting `count += 1`, creates an infinite loop.

Loop Control Statements

Continue
Skip the rest of the code and proceed to the next cycle

Break
Break out of the loop and halt iteration

Pass
Proceed with the next step as usual

Loop Control Statements

Continue Statement

Example:

```
for i in range(5):
if i == 3:
continue # Skip 3
print(i)
```

Output:

```
0
1
2
4
```

Nested Loops: Loops Within Loops

In nested loops, the inner loop completes fully before the outer loop advances.

```
for i in range(1, 4): # Outer loop (rows)
for j in range(1, 3): # Inner loop (columns)
print(i, j)
```

Total iterations = outer loop count × inner loop count = 3 × 2 = 6

Output:

```
1 1
1 2
2 1
2 2
3 1
3 2
```

Iterating with enumerate()

- The enumerate() function adds a counter to an iterable, returning (index, item) pairs for position-aware iteration.

```
colors = ["red", "green", "blue"]

for index, color in enumerate(colors):
print(index, color)
```

- You can specify a different starting index:

```
colors = ["red", "green", "blue"]

# Start counting from 1 instead of 0
for index, color in enumerate(colors, 1):
print(index, color)
```

Mastering Python Loops

Key Takeaways

Choose the right loop

Use for loops `for` known sequences and `while` loops when iterations depend on a condition

Special iterators

Use `range()` for number sequences and `enumerate()` to access indices and values

Microsoft Autogen

Use `break`, `continue` and `pass` to manage loop execution

Nested loops

Combine loops for multi-dimensional data, while considering performance and complexity

Building a Simple Chatbot in Python

Building a Simple Chatbot in Python

Objective: Make programmes interactive using conditionals and loops

Scenario: Build a chatbot that greets, answers queries and ends with "bye”

Building a Simple Chatbot in Python

Key Concepts Explained

- Use a `while loop` to keep the chatbot active for multiple inputs
- Use `if-elif-else` conditionals to decide responses based on user input
- Combine logic and repetition to build real-world applications

Building a Simple Chatbot in Python

```python
print("Welcome to ChatBot! (type 'bye' to exit)")

while True: # Loop keeps chatbot running
user_input = input("You: ").lower() # Take user input in lowercase

if user_input == "hi" or user_input == "hello":
print("Bot: Hello there! How can I help you?")
elif user_input == "how are you":
print("Bot: I'm just a program, but I'm doing great! How about you?")
elif user_input == "help":
print("Bot: Sure! You can ask me about 'time', 'name', or say 'bye' to exit.")
elif user_input == "time":
from datetime import datetime
print("Bot: Current time is", datetime.now().strftime("%H:%M:%S"))
elif user_input == "name":
print("Bot: I'm a simple Python chatbot created with loops and conditionals.")
elif user_input == "bye":
print("Bot: Goodbye! Have a nice day.")
break # Exit loop
else:
print("Bot: Sorry, I don't understand that. Try 'help'.")
```

Recap

Python Fundamentals

Recap

Core data types	Mastered Python's core building blocks: variables, numbers, strings, lists and dictionaries
Variable operations	Learned to declare variables, convert data types and manage user input/output.
Data Structures	Practiced creating and manipulating lists and dictionaries for efficient data organisation and access
Logic & Control Flow	Gained proficiency in writing logic-based branching statements and implementing loops with conditions

Logic Flows in Everyday Life

Programming Mirrors Daily Decisions and Repetition

Conditional logic
If it is raining, take an umbrella

Loops
Process all emails until the inbox is empty

Branching
If the traffic is heavy, take the metro; otherwise, drive

The logic for navigating daily life is the same for writing effective code.

Python User-Defined Functions:

Making code reusable, organised and efficieent

Function

Reusable Blocks Performing Specific Tasks

Promotes code reusability and modularity

Simplifies complexity by breaking problems into smaller tasks

Reduces redundancy and enhances code readability

Purpose of Functions

Without Functions

- Repeat code unnecessarily
- Struggle to maintain logic
- Face difficulty debugging
- Fail to scale effectively

With Functions

- Reuse code across tasks
- Organise related operations
- Improve code readability

Built-in v. User-Defined Functions

Built-in Functions

Provided by Python and ready to use.

```
print("Hello world!")  # Output text
len([1, 2, 3])        # Count items (returns 3)
max(10, 5, 20)        # Find maximum (returns 20)
round(3.14159, 2)     # Round number (returns 3.14)
```

Python includes over 60 built-in functions.

Built-in v. User-Defined Functions

User-Defined Functions

- Created by the user for specific tasks.
- Tailor logic to exact needs
- Extend Python's capabilities
- Encapsulate problem-solving steps

Anatomy of a Function

```python
def function_name(parameters):

"""
Docstring (optional): describes the function

"""
# function body - your code goes here
return value # optional
```

- **Def keyword:** Indicate the start of a function definition
- **Function_name:** Identify the function using a descriptive name
- **Parameters:** Provide optional inputs for the function
- **Return statement:** Send output back to the caller (optional)

Introducing Functions in Python

```python
def greet():
print("Hello! Welcome to Python.")


# Function calls
greet()
```

1. **Define** the function once
2. **Call** it as many times as needed

```
Hello! Welcome to Python.Hello! Welcome to Python.
```

Function Parameters and Arguments

Defines parameters as placeholders for data

Provides arguments as actual values in a call

Places parameters in the function header

Supports default, keyword and arbitrary arguments

Function Parameters and Arguments

Example

```
def greet(name):
    print(f'Hello, {name}!')

greet('Alice') # Output: Hello, Alice!
```

Returning Values from Functions

- Uses the return statement to pass results back to the caller
- Return statement exits the function and optionally sends back a value
- If no return statement is used, the function returns None

Returning Values from Functions

Example

```
def add(a, b):
    return a + b

result = add(3, 5)  # Output: 8
```

Multiple Parameters & Return Values

- **Multiple parameters**

```python
def add_numbers(a, b):
return a + b

result = add_numbers(5, 7)
print(result) # Output: 12
```

- **Multiple return values**

```python
def calculate(a, b):
return a + b, a - b, a * b


# Function call
s, d, p = calculate(10, 5)
print(s, d, p) # Output: 15 5 50
```

Python functions handle multiple inputs and return multiple outputs as tuples.

Default Parameters

Example:

```python
def greet(name="Guest"):
    print(f"Hello, {name}!")


# Function calls
greet()          # Uses the default value
greet("Amit")    # Uses the provided argument
```

Output:

```
Hello, Guest!Hello, Amit!
```

Default parameters add flexibility and simplify function calls.

Default Parameters

Key Benefits

- Makes parameters optional
- Provides sensible defaults
- Reduces errors
- Simplifies function calls

*args (Variable Positional Arguments)

Define with asterisk

Double asterisk (**) collects multiple keyword arguments into a dictionary.

Pass any number of arguments

Functions can process an unspecified number of arguments

Access as a tuple

Inside, args is a tuple of all passed values.

*args (Variable Positional Arguments)

Example

```
def add_all(*nums):
return sum(nums)

add_all(1, 2, 3, 4) # Returns 10
```

**kwargs (Variable Keyword Arguments)

Use double asterisks (**) to collect keyword arguments into a dictionary:

```
def show_details(**info):
for key, value in info.items():
print(f"{key}: {value}")


show_details(name="Amit", city="Hyderabad")
```

Using if __name__ == "__main__"

- **Run code conditionally:** Runs specific code blocks only if the script is the main program
- **Create reusable modules:** Enables files to function as standalone programs or reusable modules
- **Prevent Automatic Execution:** Code inside the if block won't run when imported

```
def greet():
print("Hello!")

if __name__ == "__main__":
greet()
```

Scope of Variables in Functions

Local v. Global Scope

Local scope	Global scope
Variables declared inside a function, accessible only within that function	Variables declared inside a function, accessible only within that function

Scope of Variables in Functions

Example

```
def my_function():
    local_var = 10  # Local scope

global_var = 20  # Global scope

my_function()
print(global_var)  # Output: 20
# print(local_var)  # Error: NameError
```

Exercise: Ticket Pricing Function

Accepts a person's age as its input parameter

Determines the ticket price using conditional logic

- Ages under 12: ₹100
- Ages 12 to 59: ₹200
- Ages 60 and above: ₹150

Outputs the calculated ticket price

Lambda Functions

Lambda functions are small, anonymous functions defined with lambda, ideal for short, one-time use.

Example :

```
square = lambda x: x ** 2
print(square(4))  # Output: 16
```

Best Practices

Add docstrings

Use docstrings to describe your function's purpose, parameters, and return value

Use clear names

Prefer clear parameter names like user_age over vague ones like x

Keep functions short

Design functions to be focused and concise; avoid exceeding a single screen of code

Write test cases

Verify edge cases and unexpected inputs to ensure robust function performance

Example: Chatbot with Function

Chatbot: Backend Code Example

```python
# Simple chatbot using functions

def get_response(user_input):
"""Return a chatbot response based on user input"""
user_input = user_input.lower()


if "hello" in user_input or "hi" in user_input:
return "Hello! How can I help you today?"
elif "bye" in user_input:
return "Goodbye! Have a great day!"
elif "price" in user_input:
return "Our product prices start at ₹499."
else:
return "I'm not sure I understand. Can you rephrase?"

# Chatbot loop
print("Welcome to SimpleBot! (type 'bye' to exit)")

while True:
user_message = input("You: ")
response = get_response(user_message)
print("Bot:", response)

if "bye" in user_message.lower():
break
```

Chatbot: Backend Code Example

Process

Get_response function

- Encapsulates all reply logic
- Determines the response using conditional checks

Main loop

- Continuously accepts user input
- Invokes get_response(user_message)
- Outputs the generated reply
- Terminates on the "bye" command

Reflection Activity

Real-World Logic in Daily Life

Take a minute
Identify a specific example of a decision or repetitive process from your experience.

Make it concrete
Frame it as an if-statement or loop, noting conditions and resulting actions.

Share
Record or share your example to reinforce the connection between real-world logic and code.

Q & A

Thank you